

Lab 4 Exam

准备工作：创建并切换到 lab4-exam 分支

请在自动初始化分支后，在开发机依次执行以下命令：

```
cd ~/学号
git fetch
git checkout lab4-exam
```

初始化的 lab4-exam 分支基于课下完成的 lab4 分支，并且在 tests 目录下添加了 lab4_systrace 样例测试目录。

题目背景

在实现系统调用时，除了需要保证功能正确之外，我们还希望确认：一个普通系统调用，是否真的按照 "用户态包装函数 → msyscall → 内核分发 → sys_* → 返回用户态" 的整体链路被正确执行。

本题围绕 MOS 的页面管理机制，要求各位同学新增一个**双参数系统调用**，用于查询两个虚拟地址对应页面的物理页引用计数，并返回一个组合结果。该系统调用本身并不复杂，但能够覆盖一条完整的普通系统调用链路，并检验各位同学是否真正理解：

- 当前进程地址空间中的虚拟页与物理页映射关系；
- 物理页引用计数 pp_ref 的含义；
- 普通系统调用的参数传递与返回值传递过程。

本题**不涉及**页写入异常、TLB Mod、COW、异常处理栈、IPC、fork 等内容，大家不需要考虑的太过复杂。

任务：新增系统调用 sys_trace_ref2

你需要添加一个系统调用：

```
int sys_trace_ref2(u_int a, u_int b);
```

对于该系统调用的返回值，我们首先定义一个辅助函数 ref(x)，其中 x 是一个虚拟地址，ref(x) 的定义如下：

- 若 x 不属于当前进程的合法用户地址范围，则 $\text{ref}(x) = -E_INVAL$ ；
- 若 x 属于合法用户地址范围，但其所在页在当前进程地址空间中**不存在有效映射**，则 $\text{ref}(x) = 0$ ；
- 若 x 对应页面已映射，则 $\text{ref}(x)$ 等于该物理页的引用计数 pp_ref。

则该系统调用的返回值定义为：

```
ref(a) + 2 * ref(b)
```

实现参考

你需要修改如下位置：

1. 在 `include/syscall.h` 中新增一个枚举值 `SYS_trace_ref2`；
2. 在 `kern/syscall_all.c` 中增加一个系统调用实现，并将其注册到 `syscall_table` 中：

```
int sys_trace_ref2(u_int a, u_int b) { ... }
```

3. 在 `user/include/lib.h` 中增加一个用户态系统调用声明：

```
int syscall_trace_ref2(u_int a, u_int b);
```

4. 在 `user/lib/syscall_lib.c` 中增加它的具体实现，你可以通过 `msyscall` 发起系统调用。

注意事项

- 请确保用户态函数名为 `syscall_trace_ref2`，系统调用号名称为 `SYS_trace_ref2`；
- 该系统调用共有 **2 个参数**，每个参数都代表一个虚拟地址，对于每个虚拟地址，请注意区分题目要求的三种情况；
- 查询虚拟地址 `va` 对应物理页的引用计数时，可先在当前进程页表中查找 `va` 是否已映射到某个物理页，若查找到对应页，再读取该物理页描述结构中的引用计数字段即可；
- 判断地址是否合法时，可以结合 MOS 的地址空间布局来考虑，我们这里将小于 `ULIM` 的地址视为合法用户地址范围，其中 `ULIM` 是用户空间与内核空间的边界；因此，本题中合法用户地址的上界应当取为 `ULIM`；
- 该系统调用查询的是**当前进程**地址空间，因此不需要传入 `envid`；
- 该系统调用没有副作用，只用于校验系统调用链路中的参数传递、页查询逻辑和返回值传递是否正确，大家在实现的时候不需要考虑的太复杂。

样例输出与本地测试

有如下用户程序样例：

```
#include <error.h>
#include <lib.h>
#include <mmu.h>
#include <syscall.h>

#define A ((void *) (USTACKTOP - 20 * PAGE_SIZE))
#define B ((void *) (USTACKTOP - 21 * PAGE_SIZE))
#define C ((void *) (USTACKTOP - 22 * PAGE_SIZE))
#define U ((void *) (USTACKTOP - 23 * PAGE_SIZE))

static int direct_trace_ref2(u_int a, u_int b) {
    return msyscall(SYS_trace_ref2, a, b, 0, 0, 0);
}

int main() {
    u_int self;
    int init_uu;
```

```

int init_invalid_first_ok;
int pair_ab_2;
int ref_a_3;
int ref_b_3;
int ref_c_3;
int direct_ab;
int wrap_bc;
int after_ab;
int after_c;
int edge_last_user_ok;
int sample_sum;
int r;

self = syscall_getenvid();

init_uu = syscall_trace_ref2((u_int)U, (u_int)U);
debugf("^_^ init_uu = %d\n", init_uu);
init_invalid_first_ok = syscall_trace_ref2(ULIM, (u_int)U) == -E_INVAL;
debugf("^_^ init_invalid_first_ok = %d\n", init_invalid_first_ok);

r = syscall_mem_alloc(self, A, PTE_D);
debugf("^_^ alloc_a_r = %d\n", r);
debugf("^_^ a_first = %d\n", syscall_trace_ref2((u_int)A, (u_int)U));

r = syscall_mem_map(self, A, self, B, PTE_D);
debugf("^_^ map_a_b_r = %d\n", r);
pair_ab_2 = syscall_trace_ref2((u_int)A, (u_int)B);
debugf("^_^ pair_ab_2 = %d\n", pair_ab_2);

r = syscall_mem_map(self, A, self, C, PTE_D);
debugf("^_^ map_a_c_r = %d\n", r);
ref_a_3 = syscall_trace_ref2((u_int)A, (u_int)U);
ref_b_3 = syscall_trace_ref2((u_int)B, (u_int)U);
ref_c_3 = syscall_trace_ref2((u_int)C, (u_int)U);
debugf("^_^ ref_a_3 = %d\n", ref_a_3);
debugf("^_^ ref_b_3 = %d\n", ref_b_3);
debugf("^_^ ref_c_3 = %d\n", ref_c_3);

direct_ab = direct_trace_ref2((u_int)A, (u_int)B);
wrap_bc = syscall_trace_ref2((u_int)B, (u_int)C);
debugf("^_^ direct_ab = %d\n", direct_ab);
debugf("^_^ wrap_bc = %d\n", wrap_bc);

r = syscall_mem_unmap(self, B);
debugf("^_^ unmap_b_r = %d\n", r);
after_ab = syscall_trace_ref2((u_int)A, (u_int)B);
after_c = syscall_trace_ref2((u_int)C, (u_int)U);
debugf("^_^ after_ab = %d\n", after_ab);
debugf("^_^ after_c = %d\n", after_c);

edge_last_user_ok = syscall_trace_ref2(ULIM - 1, (u_int)U) == 0;
debugf("^_^ edge_last_user_ok = %d\n", edge_last_user_ok);

sample_sum = init_uu + init_invalid_first_ok + syscall_trace_ref2((u_int)A,
(u_int)U) +

```

```

        pair_ab_2 + direct_ab + wrap_bc + after_ab + after_c +
edge_last_user_ok;
    debugf("^_^ sample_sum = %d\n", sample_sum);

    r = syscall_mem_unmap(self, A);
    debugf("^_^ unmap_a_r = %d\n", r);
    debugf("^_^ ref_a_0 = %d\n", syscall_trace_ref2((u_int)A, (u_int)U));
    debugf("^_^ ref_c_1 = %d\n", syscall_trace_ref2((u_int)C, (u_int)U));

    r = syscall_mem_unmap(self, C);
    debugf("^_^ unmap_c_r = %d\n", r);
    debugf("^_^ final_c = %d\n", syscall_trace_ref2((u_int)C, (u_int)U));

    return 0;
}

```

应当输出：

```

^_^ init_uu = 0
^_^ init_invalid_first_ok = 1
^_^ alloc_a_r = 0
^_^ a_first = 1
^_^ map_a_b_r = 0
^_^ pair_ab_2 = 6
^_^ map_a_c_r = 0
^_^ ref_a_3 = 3
^_^ ref_b_3 = 3
^_^ ref_c_3 = 3
^_^ direct_ab = 9
^_^ wrap_bc = 9
^_^ unmap_b_r = 0
^_^ after_ab = 2
^_^ after_c = 2
^_^ edge_last_user_ok = 1
^_^ sample_sum = 32
^_^ unmap_a_r = 0
^_^ ref_a_0 = 0
^_^ ref_c_1 = 1
^_^ unmap_c_r = 0
^_^ final_c = 0

```

你可以使用：

```
make test lab=4_systrace && make run
```

在本地测试上述样例（调试模式），或者使用：

```
MOS_PROFILE=release make test lab=4_systrace && make run
```

在本地测试上述样例（开启优化）。

样例说明

本题所有测试都围绕同一个系统调用 `sys_trace_ref2` 展开，评测数据会从不同角度检查：

1. 该系统调用从用户态到内核态的链路是否正确；
2. 当前进程页表中的映射查询逻辑是否正确；
3. 非法地址、未映射地址、已映射地址三种情况是否被正确区分。

请注意，本题测试的评测数据可能不仅会通过你写好的用户态包装函数发起系统调用，也可能会直接通过 `msyscall` 校验系统调用号、参数传递和内核分发过程。因此，你需要确保：

- 系统调用号正确；
- 用户态包装函数正确；
- 内核中的系统调用实现正确；
- `syscall_table` 中的分发正确。

提交评测与评测标准

请在开发机中执行下列命令后，在课程网站上提交评测：

```
cd ~/学号
git add -A
git commit -m "message" # 请将 message 改为有意义的信息
git push
```

在线评测时，所有的 `.mk` 文件、所有的 `Makefile` 文件、`init/init.c` 以及 `tests/` 和 `tools/` 目录下的所有文件都可能被替换为标准版本，因此请不要在这些文件中编写实际功能所依赖的代码。

测试点和分数说明

测试点序号	评测内容	分数
1	弱测：和样例完全相同。	10
2	弱测：检查 <code>SYS_trace_ref2</code> 的基本计算功能，不涉及边界判断。	20
3	中测：检查 <code>SYS_trace_ref2</code> 的基本计算功能，不涉及边界判断。	20
4	强测：检查 <code>SYS_trace_ref2</code> 在边界值、非法或未映射地址、页内与跨页访问下的行为。	20
5	强测：综合检查 <code>SYS_trace_ref2</code> 在动态映射变化下的整体行为与结果一致性。	30

测试点依赖关系

箭头方向表示依赖方向。

